1. **Runtime**
   Definition: Runtime is the phase when a program is actually running and executing instructions on the computer.
   Example: User input is read and processed while the program is running.
2. **Compiletime**
   Definition: Compiletime is the phase when source code is translated into machine code before the program runs.
   Example: Syntax errors like a missing semicolon are caught during compilation.
3. **OOP (Object-Oriented Programming)**
   Definition: OOP is a programming approach that organizes code into objects that contain data and behavior together.
   Example: A `Car` object has data like `speed` and functions like `accelerate()`.
4. **Encapsulation**
   Definition: Encapsulation is hiding internal data and allowing access only through controlled methods.
   Example: Making a variable `private` and accessing it using `get()` and `set()` functions.
5. **Abstraction**
   Definition: Abstraction shows only essential features and hides unnecessary implementation details.
   Example: Using a `drive()` function without knowing how the engine works internally.
6. **Inheritance**
   Definition: Inheritance allows one class to reuse and extend the properties of another class.
   Example: A `Developer` class inheriting common fields from an `Employee` class.
7. **Polymorphism**
   Definition: Polymorphism means one interface can represent different behaviors.
   Example: The same function name `draw()` behaving differently for `Circle` and `Square`.
8. **Runtime Polymorphism**
   Definition: Runtime polymorphism is method selection decided during program execution using base class pointers.
   Example: A base pointer calling a derived class's overridden function.
9. **Compiletime Polymorphism**
   Definition: Compiletime polymorphism is method selection decided at compile time

using function overloading or operators.
Example: Multiple `add()` functions with different parameter types.

10. **Use of `virtual` keyword in C++**
    Definition: The `virtual` keyword ensures the correct overridden function is called based on the object type at runtime.
    Example: Marking a base class function as `virtual` so the derived version runs when accessed via a base pointer.